LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 05 – Relasi Antar Kelas



CUT GEUBRINA DWI MAHARANI

103132400034

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dasar teori utama yang digunakan dalam praktikum ini adalah mengenai Hubungan antar Kelas. Menguji tiga jenis relasi antar kelas dalam OOP, yaitu: Komposisi, bentuk asosiasi yang menunjukkan hubungan kepemilikan mutlak (jika suatu objek dihapus oleh garbage collector, maka objek lainnya akan ikut hancur). Agregasi, bentuk asosiasi yang menggambarkan hubungan kepemilikan yang lebih longgar (objek dapat dibuat luar kelas dan tetap bisa eksistensi meskipun objek lainnya dihapus). Asosiasi, bentuk hubungan antar kelas di mana suatu objek menggunakan atau berinteraksi dengan objek lain (sebuah method menerima objek sebagai parameter untuk mengolah data struk, tanpa memiliki kepemilikan struktural terhadap objek pesanan tersebut)

B. Guided

1. [Agregrasi]

```
public class Dosen {

    private String kodeDosen;

    private String namaDosen;

//Setter

    public void setKodeDosen(String kodeDosen){

    this.kodeDosen = kodeDosen;

    }

    public void setNamaDosen(String namaDosen){

    this.namaDosen = namaDosen;

    }

    //Getter

    public String getKodeDosen(){

        return this.kodeDosen;

    }

    public String getNamaDosen(){

        return this.namaDosen;

    }
```

```
    public void giveScore(Mahasiswa s, int nilai){
        s.setNilai(nilai);
    }
    public int getScore(Mahasiswa s){
        return s.getNilai();
    }
    public static void main(String[] args) {
         Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :"+m.getNim());
        System.out.println("Nama :"+m.getNama());
        System.out.println("Nilai :"+d.getScore(m));
    }
}
```

```
ublic class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama){
    this.nama = nama;
```

```
    }
    public void setNim(String nim){
    this.nim = nim;
    }
    public String getNama(){
    return this.nama;
    }

    public String getNim(){
        return this.nim;
    }

    public void setNilai(int nilai){
        this.nilai=nilai;
    }

    public int getNilai(){
        return this.nilai;
    }
}
```

Output :

```
Nim :130118383
Nama :Budi
Nilai :90
```

2. [Agregrasi]

```java
package guided.agregrasi;


public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }
    //Behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " +
this.kodeJurusan);
        System.out.println("Nama Jurusan: " +
this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i=0; i<jumlahMahasiswa; i++){
            System.out.println(i+1 + "." +
daftarMahasiswa[i].getNIM() + "" +
daftarMahasiswa[i].getNama());
        }
    }
    //Getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
```

```
    }
    //Setter
    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
```

```
package guided.agregrasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
    }

    public String getNama(){
        return Nama;
    }
    //Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

3. [Komposisi]

```java
package guided.komposisi;


public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

```java
package guided.komposisi;

public class Monitor {
     private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

```java
package guided.komposisi;
```

```
public class Mouse {
      private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

```
package guided.komposisi;

public class Processor {
      private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

4. [Main]

```
package guided.agregrasi;


public class Jurusan {
    private static int jumlahMahasiswa;
```

```
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }
    //Behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " +
this.kodeJurusan);
        System.out.println("Nama Jurusan: " +
this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i=0; i<jumlahMahasiswa; i++){
            System.out.println(i+1 + "." +
daftarMahasiswa[i].getNIM() + "" +
daftarMahasiswa[i].getNama());
        }
    }
    //Getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
    }
    //Setter
    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }
```

```
    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
```

```
package guided.agregrasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
    }

    public String getNama(){
        return Nama;
    }
    //Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

Output:

```
Kode Jurusan: SD
Nama Jurusan: Sains Data
Daftar Mahasiswa:
1.103112400148Rendhi
2.103112400149John
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
```

C. Unguided

Soal Unguided — Modul 5: Pemrograman Berorientasi Objek

Relasi Antar Kelas: Komposisi, Agregasi, dan Asosiasi

Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint dan tugas yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya (gunakan enkapsulasi).
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Proyek wajib mengikuti konvensi penamaan standar (PascalCase untuk Class, camelCase untuk variabel/method).
- Gunakan array standar bertipe tetap (bukan `ArrayList` atau koleksi library lainnya).
- Tidak perlu menggunakan inheritance, overriding, maupun abstract class.

Soal — Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (`Main`).

Tugas 1: Implementasi Komposisi (Composition)

Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang **tidak dapat berdiri sendiri**. Siklus hidup objek internal terikat penuh pada objek utamanya.

Pada sistem cafe:

- Setiap item pesanan (`ItemPesanan`) pasti memiliki rincian harga (`RincianHarga`) yang mencatat harga dasar dan pajak (10%).
- Objek `RincianHarga` harus di-instansiasi langsung di dalam constructor `ItemPesanan`. Objek `RincianHarga` tidak pernah dibuat di `main`. Jika `ItemPesanan` dihapus, maka `RincianHarga` miliknya ikut hilang.

Kebutuhan Tugas 1

Buatlah dua class: `RincianHarga` dan `ItemPesanan`.

Field yang disarankan:

Class	Field	Jenis	Tipe	Keterangan
`RincianHarga`	`hargaDasar`	Instance	`double`	Harga awal sebelum pajak

`RincianHarga`	`pajak`	Instance	`double`	Nilai nominal pajak (10% dari harga dasar)
`ItemPesanan`	`namaItem`	Instance	`String`	Nama menu (misal: "Iced Americano")
`ItemPesanan`	`rincian`	Instance	`RincianHarga`	Objek internal rincian harga (**Komposisi**)

Method yang disarankan:

Class	Method	Return	Keterangan
`RincianHarga`	`getHargaTotal()`	`double`	Mengembalikan nilai `hargaDasar + pajak`
`ItemPesanan`	`getNamaItem()`	`String`	Mengembalikan nama item pesanan
`ItemPesanan`	`getRincian()`	`RincianHarga`	Mengembalikan objek rincian harga

Tugas 2: Implementasi Agregasi (Aggregation)

Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang **dapat berdiri sendiri**. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah.

Pada sistem cafe:

- Sebuah `Pesanan` menampung beberapa objek `ItemPesanan` menggunakan array standar.
- Objek `ItemPesanan` dibuat terpisah terlebih dahulu (di `main`), lalu dimasukkan satu per satu ke dalam `Pesanan` melalui method `tambahItem()`. Jika `Pesanan` dibatalkan atau dihapus, objek `ItemPesanan` tersebut secara konsep tetap ada.

Kebutuhan Tugas 2

Buatlah class `Pesanan`.

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
`nomorMeja`	Instance field	`int`	Nomor meja pelanggan
`daftarItem`	Instance field	`ItemPesanan[]`	Array penampung item pesanan (**Agregasi**)
`jumlahItem`	Instance field	`int`	Penanda jumlah item yang sudah masuk ke array

Method yang disarankan:

Method	Return	Keterangan
`tambahItem(ItemPesanan item)`	`void`	Memasukkan objek `ItemPesanan` dari luar ke dalam array `daftarItem`
`tampilkanPesanan()`	`void`	Menampilkan nomor meja dan seluruh item beserta harga total per item
`getNomorMeja()`	`int`	Mengembalikan nomor meja
`getDaftarItem()`	`ItemPesanan[]`	Mengembalikan array daftar item
`getJumlahItem()`	`int`	Mengembalikan jumlah item yang tersimpan

Tugas 3: Implementasi Asosiasi (Association)

Asosiasi adalah relasi interaksi antara dua objek independen yang **saling menggunakan tetapi tidak saling memiliki**. Objek tidak menyimpan objek lain sebagai field permanen, melainkan berinteraksi melalui parameter method.

Pada sistem cafe:

- Seorang `Kasir` memproses pembayaran dari sebuah `Pesanan`.
- Class `Kasir` tidak menyimpan objek `Pesanan` sebagai field, melainkan menerima objek `Pesanan` sebagai parameter pada method `prosesPembayaran()`.

Kebutuhan Tugas 3

Buatlah class `Kasir`.

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
`namaKasir`	Instance field	`String`	Nama petugas kasir

Method yang disarankan:

Method	Retur	Keterangan
`prosesPembayaran(Pesanan pesanan)`	`void`	Mengakses data `Pesanan`, menghitung akumulasi total bayar seluruh item, lalu mencetak struk pembayaran (**Asosiasi**)

Tugas 4: Eksekusi dan Pengujian di `Main`

Buatlah class `Main` untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.

Alur program di `main`

1. Buat 1 objek `Kasir` (misal: `"Budi"`).
2. Buat 3 objek `ItemPesanan` secara independen:
   - `"Iced Americano"`, harga dasar: `18000`
   - `"Beef Croissant"`, harga dasar: `25000`
   - `"Matcha Latte"`, harga dasar: `22000`
3. Buat 1 objek `Pesanan` untuk **Meja 4** (kapasitas array misal: 5 item).
4. Masukkan ketiga objek `ItemPesanan` ke dalam objek `Pesanan` menggunakan method `tambahItem()`.
5. Tampilkan rincian pesanan dengan memanggil `tampilkanPesanan()` pada objek `Pesanan`.
6. Jalankan pemrosesan pembayaran dengan memanggil method `prosesPembayaran(pesanan)` dari objek `Kasir`.

Output yang Diharapkan

```
=== DAFTAR PESANAN CAFE ===

Nomor Meja: 4

1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)

2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)

3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)


=== STRUK PEMBAYARAN KASIR ===

Kasir Bertugas : Budi

Pesanan        : Meja 4

Jumlah Item    : 3

Total Subtotal : Rp 65000.0

Total Pajak    : Rp 6500.0

TOTAL BAYAR    : Rp 71500.0

==============================
```

Yang Perlu Diperhatikan Saat Mengerjakan

- **Komposisi:** Objek `RincianHarga` wajib di-instansiasi di dalam constructor `ItemPesanan` menggunakan kata kunci new. Dilarang melakukan `new RincianHarga(...)` di dalam `main`.
- **Agregasi:** Objek `ItemPesanan` wajib di-instansiasi di `main` terlebih dahulu, kemudian dikirimkan sebagai argumen ke method `tambahItem()`.

- **Asosiasi:** Class `Kasir` dilarang memiliki field bertipe `Pesanan`. Interaksi hanya terjadi saat method `prosesPembayaran(Pesanan pesanan)` dipanggil.

Jawaban :

1. Komposisi

```java
package unguided.komposisi;

public class Produk {
    private String namaProduk;
    private RincianHarga rincian;

    //constructor
    public Produk(String namaProduk, double hargaDasar){
        this.namaProduk = namaProduk;
        this.rincian = new RincianHarga(hargaDasar);
    }

    //Getter
    public String getNamaProduk(){
        return this.namaProduk;
    }

    public RincianHarga getRincian(){
        return this.rincian;
    }
}
```

```java
package unguided.komposisi;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    //Construdstor
    public RincianHarga(double hargaDasar){
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }
```

```
    public double getHargaTotal(){
        return this.hargaDasar + this.pajak;
    }

    //Getter
    public double getHargaDasar(){
        return this.hargaDasar;
    }

    public double getPajak(){
        return this.pajak;
    }
}
```

2. Agregasi

```
package unguided.agregasi;

import unguided.komposisi.Produk;

public class Pesanan {
    private int nomorMeja;
    private Produk[] daftarProduk;
    private int jumlahProduk;

    //Constructor
    public Pesanan(int nomorMeja, int kapasitasMax){
        this.nomorMeja = nomorMeja;
        this.daftarProduk = new Produk[kapasitasMax];
        this.jumlahProduk = 0;
    }

    public void tambahProduk(Produk produk){
        if (jumlahProduk < daftarProduk.length){
            daftarProduk[jumlahProduk] = produk;
            jumlahProduk++;
        } else {
            System.out.println("Kapasistas pesanan meja" +
nomorMeja + " sudah penuh!");
```

```
        }
    }

    public void tampilkanPesanan(){
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahProduk; i++){
            Produk produk = this.daftarProduk[i];
            double totalHarga =
produk.getRincian().getHargaTotal();
            double hargaDasar =
produk.getRincian().getHargaDasar();
            double pajak = produk.getRincian().getPajak();

            System.out.println((i+1) + ". " +
produk.getNamaProduk() + " - Rp" + totalHarga + " (Harga: " +
hargaDasar + ", Pajak: " + pajak + ")");
        }
        System.out.println();
    }
    //Getter
    public int getNomorMeja(){
        return this.nomorMeja;
    }

    public Produk[] getDaftarProduk() {
        return this.daftarProduk;
    }

    public int getJumlahProduk(){
        return this.jumlahProduk;
    }
}
```

3. Asosiasi

```
package unguided.asosiasi;
```

```java
import unguided.agregasi.Pesanan;
import unguided.komposisi.Produk;

public class Kasir {
    private String namaKasir;

    //Constructor
    public Kasir(String namaKasir){
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan){
        double totalSubtotal = 0;
        double totalPajak = 0;
        Produk[] produk = pesanan.getDaftarProduk();
        int jumlah = pesanan.getJumlahProduk();

        for(int i = 0; i < jumlah; i++){
            totalSubtotal +=
produk[i].getRincian().getHargaDasar();
            totalPajak += produk[i].getRincian().getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " +
this.namaKasir);
        System.out.println("Pesanan        :  Meja" +
pesanan.getNomorMeja());
        System.out.println("Jumlah Item    :  " + jumlah);
        System.out.println("Total Subtotal : Rp" +
totalSubtotal);
        System.out.println("Total Pajak    : Rp" + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp" + totalBayar);

System.out.println("================================");

    }
```

```
    public String getNamaKasir() {
        return this.namaKasir;
    }

}
```

4. Main

```
package unguided;

import unguided.asosiasi.Kasir;
import unguided.komposisi.Produk;
import unguided.agregasi.Pesanan;

public class main {
    public static void main(String[] args){
        Kasir kasir = new Kasir("Budi");

        Produk produk1 = new Produk("Iced Americano", 18000);
        Produk produk2 = new Produk("Beef Croissant", 25000);
        Produk produk3 = new Produk("Matcha Latte", 22000);

        Pesanan pesananMeja4 = new Pesanan(4 ,5);

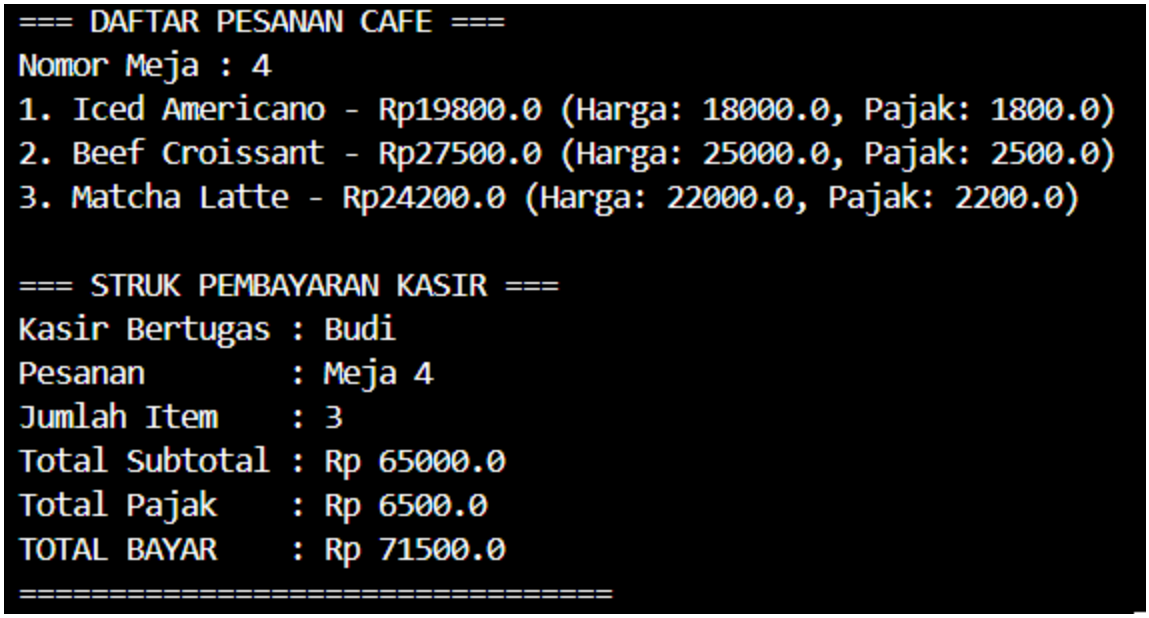
        pesananMeja4.tambahProduk(produk1);
        pesananMeja4.tambahProduk(produk2);
        pesananMeja4.tambahProduk(produk3);

        pesananMeja4.tampilkanPesanan();

        kasir.prosesPembayaran(pesananMeja4);

    }
}
```

Output :



Penjelasan :

1. Kelas Rincian Harga
   kelas ini bertugas untuk menyimpan dan mengelola detail finansial dari suatu produk. Dengan variabel hargaDasar (harga produk sebelum kena pajak) pajak (10% dari produk). Void getHargaTotal digunakan untuk menghasilkan total harga jual produk tersebut

2. Kelas Produk
   kelas ini merepresentasikan item menu yang ada di kafe. Konsep oop digunakan pada komposisi. Objek RincianHarga dibuat langsung di dalam constructor Produk. Jika objek Produk-nya dihapus, maka rincian harganya ikut hancur

3. Kelas Pesanan
   Kelas ini mencatat daftar produk yang dipesan untuk meja tertentu. Konsep OOP menggunakan Agregasi. Objek pesanan menampung kumpulan objek produk di dalam sebuah array. Produk bisa eksistensi sendiri terlepas dari pesanan. Fungsi utama yang digunakan tamabahProduk untuk memasukkan objek produk ke dalam array selama kapasitas meja masih cukup. Fungsi tampilkanPesanan berguna mencetak daftar menu yang di pesan beserta rincian harganya

4. Kelas Kasir
   Kelas ini merepresentasikan petugas kasir yang melayani transaksi. Hubungan asosiasi digunakan dimana kelas kasir menggunakan objek pesanan sebagai parameter di dalam method-nya untuk diprose. Fungsi utama prosesPembayaran

berguna mengambil data dari objek pesanan, lalu menjumlahkannya menajadi total subtotal, total pajak, dan total bayar lalu dicetak menjadi struk pembayaran

5. Kelas Main
   Pada kelas ini alurnya yaitu:
   - Membuat objek kasir beserta namanya
   - Membuat beberapa objek produk beserta harganya
   - Membuat objek pesanan untuk nomor meja 4 dengan kapasitas maksimal 5 produk
   - Memasukkan ketiga produk tadi ke dalam pesananMeja4 menggunakan method tambahProduk()
   - Memanggil method tampilkanPesanan() untuk melihat daftar pesanan di meja 4
   - Memanggil method kasir.prosesPembayaran(pesananMeja4) untuk menghitung total keseluruhan dan mencetak struk pembayaran

D. Kesimpulan
   Praktikum ini mengimplementasikan OOP, khususmya enkapsulasi untuk menjaga keamanan data di setiap kelas. Selain itu program ini juga mendemonstrasikan tiga bentuk relasi antar kelas. Komposisi diterapkan di Produk dan RincianHarga. Agregasi diterapkan antara kelas pesanan dan produk menggunakan array. Asosiasi diterapkan antara kelas kasir dan pesanan. Dengan memecah fungsionalitas program ke dalam beberapa kelas spesifik sehingga kode menjadi lebih terstruktur dan mudah dikelola